Experiment No. 4

AIM

Using the alternate Employee schema, demonstrate INNER JOIN, LEFT JOIN, self-join, 3-way join, correlated subqueries, EXISTS, simulated INTERSECT and EXCEPT. Analyze selected queries using EXPLAIN.

METHODOLOGY

1. Use the Employee–Department–Project database prepared in Experiment 3.
2. Begin with direct table relationships and use joins to progressively combine related information.
3. Use INNER JOIN when only matching records are required and LEFT JOIN when unmatched employees must also remain visible.
4. Use a self-join to compare employees working in the same department without duplicating the Employee table.
5. Use a three-table join to connect employees, departments and projects in one result.
6. Use a correlated subquery when the inner query depends on the current employee row.
7. Use EXISTS and NOT EXISTS to test membership and simulate INTERSECT / EXCEPT behavior.
8. Finish by using EXPLAIN to inspect access methods, indexes and estimated rows.

RELATIONSHIPS USED

Relationship	Join condition
Employee 🡘 Department	Employee.dept_id = Department.dept_id
Employee 🡘 Employee_Project	Employee.emp_id = Employee_Project.emp_id
Employee_Project 🡘 Project	Employee_Project.project_id = Project.project_id
Employee 🡘 Employee	Employee.dept_id = Employee.dept_id for self-join comparisons

QUERY WORKFLOW

Step	Operation
1	Retrieve employee and department information with INNER JOIN.
2	Preserve all employees with LEFT JOIN.
3	Compare employees within the same department using self-join.
4	Combine Employee, Department and Project data with a 3-way join.
5	Find employees above their department's average salary using a correlated subquery.
6	Use EXISTS and NOT EXISTS for existence checks and set-operation simulations.
7	Use EXPLAIN to inspect the execution plan of representative queries.

PROGRAM / SQL QUERIES

1. INNER JOIN – employee names with their department and location:

```
SELECT e.emp_id, e.emp_name,
d.dept_name, d.location
FROM Employee e
INNER JOIN Department d ON
e.dept_id = d.dept_id
ORDER BY d.dept_name, e.emp_name;
```

2. LEFT JOIN – retain every employee and show project details when available:

```
SELECT e.emp_name,
d.dept_name,
p.project_name FROM
Employee e
LEFT JOIN Department d ON
e.dept_id=d.dept_id
LEFT JOIN Employee_Project ep ON
e.emp_id = ep.emp_id
LEFT JOIN Project p
ON ep.project_id=p.project_id ORDER BY
e.emp_id;
```

3. Self-join – compare pairs of employees from the same department:

```
SELECT a.emp_name AS employee_a,
b.emp_name AS employee_b, d.dept_name
FROM Employee a
JOIN Employee b
ON a.dept_id=b.dept_id AND
a.emp_id < b.emp_id JOIN
Department d
ON a.dept_id = d.dept_id
ORDER BY d.dept_name, employee_a;
```

4. 3-way join – employee, project and department information:

```
SELECT e.emp_name,
d.dept_name,
p.project_name,
ep.hours_worked FROM
Employee e JOIN
Department d
ON e.dept_id=d.dept_id JOIN
Employee_Project ep ON e.emp_id =
ep.emp_id JOIN Project p
ON ep.project_id=p.project_id WHERE
ep.hours_worked >= 120 ORDER BY
ep.hours_worked DESC;
```

5. Correlated subquery – employees earning above their own department average:

```
SELECT e.emp_name,
e.dept_id, e.salary
FROM Employee e WHERE
e.salary > ( SELECT
AVG(x.salary) FROM
Employee x
WHERE x.dept_id = e.dept_id
)
ORDER BY e.salary DESC;
```

6. EXISTS – departments having at least one employee earning above 85,000:

```
SELECT d.dept_id, d.dept_name
FROM Department d
WHERE
EXISTS
( SELECT 1
FROM Employee e
WHERE e.dept_id=d.dept_id AND
e.salary > 85000
```

SET-OPERATION SIMULATIONS / OUTPUT

7. Simulated INTERSECT – employees who work on a project and belong to a department with average salary above 75,000:

```
SELECT e.emp_id, e.emp_name FROM
Employee e
WHERE EXISTS
( SELECT 1
FROM Employee_Project ep WHERE
ep.emp_id=e.emp_id
)
AND e.dept_id IN
( SELECT dept_id FROM
Employee GROUP BY
dept_id
HAVING AVG(salary) > 75000
)
ORDER BY e.emp_id;
```

8. Simulated EXCEPT – employees who are not assigned to the Predictive AI project:

```
SELECT e.emp_id, e.emp_name FROM
Employee e
WHERE NOT EXISTS
( SELECT 1
FROM Employee_Project ep JOIN
Project p
ON ep.project_id=p.project_id WHERE
ep.emp_id = e.emp_id
AND p.project_name = 'Predictive AI'
)
ORDER BY e.emp_id;
```

SAMPLE OUTPUT

Query	Example result
INNER JOIN	Employee records are paired with their department details.
LEFT JOIN	Every employee remains visible; project information appears when assigned.
Self-join	Pairs of employees from the same department are generated.
3-way JOIN	Employee, department, project and hours worked are combined.
Correlated subquery	Employees above the average salary of their own department.
EXISTS	Departments containing employees above the selected salary threshold.
INTERSECT simulation	Employees satisfying both project and department conditions.
EXCEPT simulation	Employees not associated with the selected Predictive AI project.

JOIN COMPARISON

Join type	Use in this experiment	Result behavior
INNER JOIN	Employee + Department	Only matching employee/dept rows
LEFT JOIN	Employee + Project	All employees are retained
Self-join	Employee + Employee	Pairs within the same department
3-way JOIN	Employee + Department + Project	Combined related information

OBSERVATION

The alternate queries use different filtering conditions and a different set-operation interpretation from the first version. EXISTS and NOT EXISTS are used to express membership conditions without relying on direct INTERSECT or EXCEPT syntax.

WORKING / EXPLAIN ANALYSIS

9. Inspect the execution plan for the project workload join:

```
EXPLAIN
SELECT e.emp_name, d.dept_name,
p.project_name, ep.hours_worked FROM
Employee e
JOIN Department d
ON e.dept_id=d.dept_id JOIN
Employee_Project ep ON e.emp_id =
ep.emp_id JOIN Project p
ON ep.project_id=p.project_id WHERE
ep.hours_worked >= 120 ORDER BY
ep.hours_worked DESC;
```

10. Inspect the execution plan for the correlated salary query:

```
EXPLAIN
SELECT e.emp_name, e.salary FROM
Employee e
WHERE e.salary > ( SELECT
AVG(x.salary) FROM
Employee x
WHERE x.dept_id = e.dept_id
);
```

EXPLAIN FIELDS TO OBSERVE

Field	Purpose
type	Shows the access method used for each table.
possible_keys	Lists indexes that may be useful for the query.
key	Shows the index actually selected, when applicable.
rows	Estimated number of rows examined.
Extra	Additional operations such as filtering, sorting or temporary processing.

OBSERVATIONS / OUTCOME

No.	Observation
1	Demonstrated INNER JOIN and LEFT JOIN with the alternate Employee schema.
2	Used a self-join to compare employees within the same department.
3	Combined four related tables in a project-oriented join.
4	Used a correlated subquery to compare each salary with its department average.
5	Used EXISTS and NOT EXISTS to simulate intersection and difference conditions.
6	Used EXPLAIN to inspect access methods, possible indexes and estimated rows.

RESULT

The alternate Employee–Department–Project database was used successfully to demonstrate INNER JOIN, LEFT JOIN, self-join, multi-table joins, correlated subqueries, EXISTS, simulated INTERSECT and EXCEPT, and EXPLAIN-based execution-plan analysis.

EXECUTION

```
mysql -u root -p USE
enterprise_db;

EXPLAIN SELECT ...;
```